RNSQL：融合逆规范化的 Text2SQL 生成

帖　军[1,2]　范子琪[1,2]　孙　翀[1,3*]　郑　禄[1,3]　朱柏尔[1,3]

[1](中南民族大学计算机科学学院　湖北 武汉 430074)

[2](农业区块链与智能管理湖北省工程研究中心　湖北 武汉 430074)

[3](湖北省制造企业智能管理工程技术研究中心　湖北 武汉 430074)

摘　要　Text2SQL 是自然语言处理科研领域中的一项重要任务,在研究智能问答系统中发挥关键性的作用,其核心任务是将自然语言描述的问题自动转换为 SQL 查询语句。当前研究重点为提高 SQL 子句任务的匹配准确率,但忽略了 SQL 的句法生成的正确性,涉及多表连接的 SQL 生成仍存在大量错误。因此,提出一种基于神经网络的 Text2SQL 方法,该方法通过逆规范化技术,对数据库模式进行重构,关注 SQL 句法生成的正确性,称为逆规范化网络(Reverse Normalization SQL, RNSQL)。经理论分析和在公共数据集 Spider 上实验验证,RNSQL 能有效提升 Text2SQL 任务的质量。

关键词　逆规范化　语义解析　Text2SQL　槽填充

中图分类号　TP391.1　　**文献标志码**　A　　DOI:10.3969/j.issn.1000-386x.2025.09.005

RNSQL: TEXT2SQL GENERATION BASED ON REVERSE NORMALIZATION

Tie Jun[1,2]　Fan Ziqi[1,2]　Sun Chong[1,3*]　Zheng Lu[1,3]　Zhu Boer[1,3]

[1](*College of Computer Science, South-Central Minzu University, Wuhan 430074, Hubei, China*)

[2](*Hubei Provincial Engineering Research Centre for Agricultural Blockchain and Intelligent Management, Wuhan 430074, Hubei, China*)

[3](*Hubei Provincial Engineering Research Center for Intelligent Management of Manufacturing Enterprises, Wuhan 430074, Hubei, China*)

Abstract　Text2SQL is an essential task in natural language processing scientific research. It plays a crucial role in studying intelligent question and answer systems, where the core task is to automatically convert questions described in natural language into SQL query statements. Current research focuses on improving the matching accuracy of SQL clause tasks. However, it ignores the correctness of syntactic generation of SQL, and the production of SQL involving multiple tables joining still suffers from a large number of errors. As a result, a neural network-based Text2SQL approach is proposed, which refactors the database schema to focus on the correctness of SQL syntax generation through an inverse normalization technique called RNSQL (Reverse Normalization SQL). Validated by theoretical analysis and experiments on the public dataset Spider, RNSQL can effectively improve the quality of Text2SQL tasks.

Keywords　Reverse normalization　Semantic parsing　Text2SQL　Slot filling

0　引　言

结构化查询语言(SQL)是被广泛应用于数据库的查询语言。将自然语言自动解析为 SQL 具有巨大的潜力,它能便于非专业用户使用自然语言查询和挖掘结构化数据。Text2SQL 是关系数据库智能问答系统的核心,也是自然语言处理领域中的一项重要任务。为了实现实用的 Text2SQL 工作流,模型需要将自然语言查询与给定的数据库关联起来。通过自然语言与关系数据库或表交互是一个重要且具有挑战性的问题。

Text2SQL 包含两项子任务,首先要构建正确的

收稿日期:2022-07-31。国家民委中青年英才培养计划项目(MZR20007);湖北省科技重大专项(2020AEA011);武汉市科技计划应用基础前沿项目(2020020601012267);中南民族大学研究生创新基金项目(3212021yjshq003)。**帖军**,教授,主研领域:知识发现与数据挖掘,机器感知与模式识别。**范子琪**,硕士生。**孙翀**,讲师。**郑禄**,讲师。**朱柏尔**,硕士生。

SQL 语句框架(可称为句法生成任务),在此基础上,再将框架中的每一个子元素与相应的值相匹配(可称为值匹配任务)。句法生成任务与值匹配任务并不是完全独立的任务,句法任务与值匹配任务相互影响相互促进,从而得到高质量 SQL 语句。

如图 1 所示,单轮 Text2SQL 任务以单句自然语句(Query)“List all singer names in concerts in year 2014.”、数据库模式(Schema)为输入,得到一条 SQL 查询语句为输出,其中“SELECT”“FROM”“JOIN”“WHERE”构成了 SQL 语句的句法结构部分,“T2. name”是 SQL 语句中列名匹配值,“singer_in_concert”是 SQL 语句中表名匹配值,其余部分为条件匹配值。

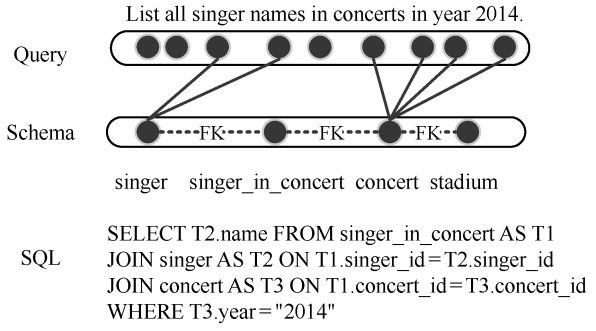

图 1 多表单轮 Text2SQL 问题示例

其中,句法问题的解决方案致力于构造合适的槽填充结构承托字符串匹配、启发式融合编码,以支持复杂本文转化为 SQL 语句模型。在主流方案中,RYANSQL[1]将句法问题转化为具体的嵌套句式问题,通过标注语句位置信息将嵌套 SQL 语句转换为非嵌套 SQL 语句,以此改进草图槽填充方法的准确性,然而表连接的选择错误仍大量存在。

究其多表连接错误的大量存在的根源,这与多表单轮问题与单表单轮问题存在巨大任务量差异有关,如列名所属表名的选择、多表连接的路径选择等一系列相关任务。与多表单轮 Text2SQL 任务相比,RYANSQL、RATSQL 等主流模型在单表单轮 Text2SQL 任务上普遍具有更高的准确率。

为优化句法结构中表连接问题,本文对句法问题在 Text2SQL 任务中的作用进行了深入探究。如 Dong 等[2]所提,从模型的输入中检测和删除特定于域的单词可以让模型专注于学习自然语言和 SQL 之间的语法转换,从而减少输入的句法稀疏性。融合数据库模式感知技术的句法改进有益于 Text2SQL 任务的生成,模式感知通过挖掘数据库模式中的信息,如表名、列名、主键、外键,融合模式感知的表连接有助于优化多表连接过程中表连接路径的选择,也对过渡连接表的模式信息干扰起到一定保护性作用。

本文提出一种基于数据库模式信息挖掘的模式逆规范化方法,用来优化 Text2SQL 技术。通过逆规范化方法,对数据库模式进行重构,从而削减数据库中模式边界,进一步用分解关联查询的方式来生成查询。重构后的数据库模式通过神经网络“编码-解码”。逆规范化的优势不仅体现在减少连接操作的需求、减少数据库模式中索引和外键的数量,更是加快查询 SQL 生成的速度。

1 相关工作

Text2SQL 在深度神经网络上的应用主要依赖于“编码-解码”框架。“编码-解码”框架中的编码部分,其任务是将输入序列转化成向量,即将自然语言问句及数据库模式信息转化成定长向量;而“编码-解码”框架中的解码部分,其任务则是将生成向量转化成输出序列,即将定长向量转化成生成的 SQL 查询语句。“编码-解码”框架应用广泛,在发展早期被应用于机器翻译、文本生成等领域,继而应用在文档摘取、问答系统等领域。在编码器的选择上,编码器发展经历了基于长短期记忆网络(LSTM)的编码器、基于图神经网络(GNN)的编码器以及基于 BERT 的编码器等编码器技术演变。解码器的研究主要围绕序列生成,槽填充以及语法生成三种方式。

RYANSQL 提出一种基于草图的槽填充方法来生成 SQL 语句,将嵌套的 SQL 查询转换为一组非嵌套的 SELECT 语句,用于复杂的 Text2SQL 问题。RYANSQL 提出了复杂 SELECT 语句预测的详细草图,以及处理嵌套查询的语句位置代码。此外,还提出两种输入操作方法进一步提高系统整体性能。在具有挑战性的 Spider 基准数据集上实现了最先进的性能。误差分析表明,应该根据其他槽的预测结果来更新槽值。

基于草图的时隙填充方法,首先由 Xu 等[3]提出,用于处理 WikiSQL 数据集,使用用于 SQL 查询的时隙定义草图,解码器对这些时隙的值进行分类。Hwang 等[4]和 He 等[5]在 BERT[6]的帮助下,在 WikiSQL 数据集上实现了基于草图的方法最先进的性能。然而,与基于语法的方法相比,基于更复杂的 Spider 基准的草图方法[7]的性能相对较低。有两个主要原因:(1) 很难为 Spider 查询定义草图,因为 Spider 中 SQL 查询允许的语法远比 WikiSQL 查询复杂。(2) 由于基于槽填充的方法会为预定义的插槽填充值,因此这些方法在预测嵌套查询时存在困难。

模式链接的引入是新的研究重点之一,如知识图谱

领域的实体链接[8-10]和对话系统中的缓慢填充[11-12],其中提出了大量注释数据和模型,以解决其特定属性。在语义分析的一般领域,已经证明将底层结构与词汇解耦有利于跨领域语义分析[13]。然而,当涉及到文本到 SQL 问题时,现有方法常将模式链接视为使用简单启发式的次要预处理过程,例如自然语言语句和列或表名称之间的字符串匹配[14]。简单的启发式方法很难准确识别自然语言话语中涉及的列或表,也很难理解话语与相应数据库模式之间的关系。因此,他们将图式链接作为一个单独的研究问题迈出了第一步。然而,由于缺乏直接的模式链接监督,它们在具有挑战性的 Spider 数据集上实现了有限的改进,进一步说明了这个问题的困难。与之前的方法不同,最近的模型[15-16]将模式链接集成为可学习的方法。

本文提出了一种基于逆规范化的 Text2SQL 方法,引入逆规范化技术,将数据库模式链接起来,构建"编码-解码式"语义解析器 RNSQL,通过逆规范化将多表连接查询问题简化为单表查询问题,减少表的查找与连接,从而提高查询准确率,方案质量得到有效提升。

2　问题定义

Text2SQL 任务形式化表示为:

$$sql_q = text2SQL(q, D) \tag{1}$$

式中:输入由自然语言语句 q 及数据库模式集合 D 组成,输出 sql_q 表示 SQL 语句,数据库模式集合 D 包含了表名、列名、主键标记、外键关系等信息,其中 T 表示数据库中的表名集合,C 表示数据库中的列名集合,PK 表示主键标记集合,FK 表示外键关系集合。

本文工作可形式化表示为:

$$f = \max \sum_{\forall q_i \in Q} G(sql_{q_i} \mid sql_{q_i} = text2SQL(q, D), sql_{q_i'} \mid sql_{q_i'} = text2SQL(q, D^C)) \tag{2}$$

式中:D^C 为逆规范化后的数据库模式,sql_{q_i}为逆规范化后生成的 SQL 语句,RN 为逆规范化过程,其形式化表现为式(3)。

$$D^C = RN(D) \tag{3}$$

式(2)中 G 函数定义如式(4)所示。

$$G(Z_i, Z_i') = \begin{cases} 1 & Z_i' - Z_i > 0 \\ 0 & Z_i' - Z_i \leqslant 0 \end{cases} \tag{4}$$

式中:Z_i'表示逆规范化后综合评估值,Z_i 表示未经逆规范化过程综合评估值,其中,函数 G 表示逆规范化后与逆规范化前的 SQL 语句精确准确率之差,$Z_i = EMA(sql_d)$,当 $EMA(sql_d) > 0$ 时,则 Z_i 取正值,当 $EMA(sql_d) = 0$ 时,Z_i 取 0。EMA 为 SQL 语句准确率评估函数,定义为式(5)。

$$EMA(sql_d) = \begin{cases} 1 & sql_d = sql_{\text{d-gold}} \\ 0 & sql_d! = sql_{\text{d-gold}} \end{cases} \tag{5}$$

当 $EMA(sql_d) = 1$ 时,此时逆规范化后的 SQL 语句输出正确,当 $EMA(sql_d) = 0$ 时,此时逆规范化后的 SQL 语句输出错误。本文工作旨在求得使式(2)中任意输入函数 G 之和最大,即令 SQL 查询语句生成质量提升最优的 RN 模型。

3　数据库模式逆规范化

为深入研究数据库模式在多表连接上的作用,本文提出 RNSQL 模型,模型基于编码-解码框架,模型编码部分如图 2 所示,包括一个五层结构输入编码器基本模型,与一个逆规范化模式重构组件(RN),解码部分由表 1 所示十层填充槽构成。

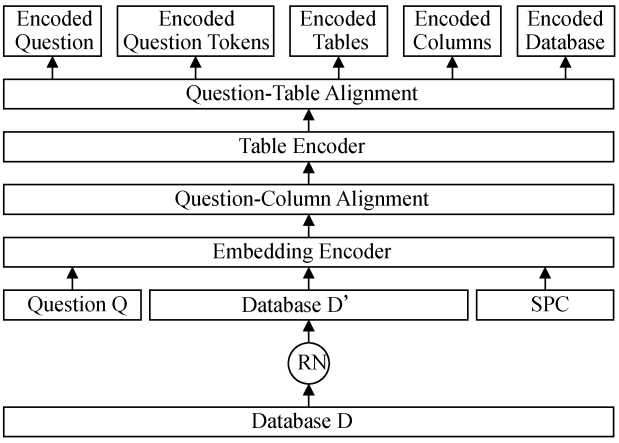

图 2　输入编码器结构

表 1　子任务与对应填充槽

子任务	填充槽
FROM	($TBL) +
SELECT	$DIST($AGG($DIST1 $agg1 $col1 $ARI $DIST2 $AGG2 $COL2)) +
ORDERBY	(($DIST1 $AGG1 $COL1 $ARI $DIST2 $AGG2 $COL2) $ORD) *
GROUPBY	($COL) *
LIMIT	$NUM
WHERE	($CONJ($DIST1 $AGG1 $COL1 $ARI $DIST2 $AGG2 $COL2)
HAVING	$NOT $COND $VAL1 | $SEL1 $VAL2 | $SEL2) *
INTERSECT	$SEL
UNION	
EXCEPT	

3.1 基本模型

基本模型基于编码-解码框架,编码部分由嵌入层、编码嵌入层、问题列对齐层、表编码层与问题表对齐层联结,解码部分由依据 SQL 句法设计的十层填充槽构成。

嵌入层基于 BERT 编码器,其输入由数据库模式(Database D)、位置信息代码(SPC)与自然语言问题(Q)组成,如下所示:[CLS],W_1^Q,[SEP],C_j^P,[SEP],$W_K^{C_1}$,[SEP],…,$W_K^{C_m}$,[SEP]在 BERT 编码器输入向量中,[CLS]标志符置于第一个句子的首位,通过 BERT 编码后得到的表征向量能够使用于后续分类子任务,而[SEP]标志符则用于分开两条输入语句。

嵌入编码层引入 Glove,一种用于捕捉语料词汇间的语义特征的词表征工具,Glove 将语料库词汇表示为实数构成的向量表征,不仅引入词频特征,更是引入语料库词汇上下文局部特征,以抓取语料库词汇之间的相似性。

位置信息代码在嵌入编码层由嵌入向量$\{\boldsymbol{e}_1^{\mathrm{p}},\boldsymbol{e}_2^{\mathrm{p}},\cdots,\boldsymbol{e}_p^{\mathrm{p}}\}$经过最大池化层得到位置信息代码语义向量 $\boldsymbol{V}^{\mathrm{P}}$,将 $\boldsymbol{V}^{\mathrm{P}}$ 与问题及模式信息嵌入向量连接,利用密集卷积神经网络对问题和数据库模式中的每个单词编码,深度卷积层捕获相对位置信息。

在列编码层模型使用 DenseCNN 对问题和列名中的单词进行编码,以捕获语义特征,输出问题词编码 $\boldsymbol{V}^{\mathrm{Q}}=\{\boldsymbol{v}_1^{\mathrm{Q}},\boldsymbol{v}_2^{\mathrm{Q}},\cdots,\boldsymbol{v}_n^{\mathrm{Q}}\}\in\mathbf{R}^{n\times d}$及隐藏列向量 $\boldsymbol{H}^{\mathrm{C}}=\{\boldsymbol{h}_1^{\mathrm{C}},\boldsymbol{h}_2^{\mathrm{C}},\cdots,\boldsymbol{h}_m^{\mathrm{C}}\}\in\mathbf{R}^{m\times d}$。

在问题列对齐层,将问题标记与相应的列联系起来,用问题的上下文信息对列向量进行建模。用缩放的点积注意将问题标记与列向量对齐。

$$\boldsymbol{A}_{\mathrm{Q2C}}=\mathrm{softmax}\left(\frac{\boldsymbol{H}^{\mathrm{c}}*(\boldsymbol{V}^{\mathrm{q}})^{\mathrm{T}}}{\sqrt{d}}\right)\times\boldsymbol{V} \tag{6}$$

对于数值较大的缩放因子 d 值,点积呈现数量级的大幅增长,将 softmax 函数推向具有极小梯度的区域。为了减小点积大幅增长带来的影响,对点积做扩展,如式(6)所示。对 A 做启发式融合,得到融合列矩阵 $\boldsymbol{F}^{\mathrm{C}}$,在融合列矩阵 $\boldsymbol{F}^{\mathrm{C}}$ 上应用 Transformer,以捕获上下文列信息,输出列编码向量 $\boldsymbol{V}^{\mathrm{C}}=\{\boldsymbol{v}_1^{\mathrm{C}},\boldsymbol{v}_2^{\mathrm{C}},\cdots,\boldsymbol{v}_m^{\mathrm{C}}\}$。

表编码层将每张表的列向量积分以获得编码的表向量 $\boldsymbol{H}^{\mathrm{T}}=\{\boldsymbol{h}_1^{\mathrm{T}},\boldsymbol{h}_2^{\mathrm{T}},\cdots,\boldsymbol{h}_t^{\mathrm{T}}\}\in\mathbf{R}^{t\times d}$。在问题表对其层,与问题列对其层具有相同的网络体系结构,用于问题语句中上下文信息对表向量建模,输出表编码向量 $\boldsymbol{V}^{\mathrm{T}}=\{\boldsymbol{V}_1^{\mathrm{T}},\boldsymbol{V}_2^{\mathrm{T}},\cdots,\boldsymbol{V}_t^{\mathrm{T}}\}$。

输入编码器的最终输出是问题词编码 $\boldsymbol{V}^{\mathrm{Q}}=\{\boldsymbol{v}_1^{\mathrm{Q}},\boldsymbol{v}_2^{\mathrm{Q}},\cdots,\boldsymbol{v}_n^{\mathrm{Q}}\}$,列编码 $\boldsymbol{V}^{\mathrm{C}}=\{\boldsymbol{v}_1^{\mathrm{C}},\boldsymbol{v}_2^{\mathrm{C}},\cdots,\boldsymbol{v}_m^{\mathrm{C}}\}$,表编码 $\boldsymbol{V}^{\mathrm{T}}=\{\boldsymbol{V}_1^{\mathrm{T}},\boldsymbol{V}_2^{\mathrm{T}},\cdots,\boldsymbol{V}_t^{\mathrm{T}}\}$,位置编码 $\boldsymbol{V}^{\mathrm{P}}$,问题编码 $\boldsymbol{V}^{\mathrm{Q}}=f_{\mathrm{s}}(\boldsymbol{V}^{\mathrm{Q}})$,及数据库模式编码 $\boldsymbol{V}^{\mathrm{D}}=f_{\mathrm{s}}(\boldsymbol{V}^{\mathrm{C}})$。

SQL 语言遵守特定的规范和准则,称之为 SQL 语法。SQL 语法结构包括子句、表达式、谓词、查询及语句。依据 SQL 语法将填充槽分为十个不同子任务排列的关键任务,分别为 SQL 中 FROM 任务、SELECT 任务、ORDER BY 任务、GROUP BY 任务、LIMIT 任务、WHERE 任务、HAVING 任务、INTERSECT 任务、UNION 任务与 EXCEPT 任务。

关键任务由子任务的排列组合组成,定义如下。$SEL 和 $COL 分别代表表名与列名,$AGG 表明运算关系,表示{none,max,min,count,sum,avg}其中之一,$ARI 表明运算符,表示{none, -, +, *, /}之一,$COND 表明条件运算符,表示{between, =, >, <, >=, <=, !=, in, like, is, exists}之一。$DIST 和 $NOT 表名布尔变量,分别表示关键字 Distinct 和 NOT。$ORD 表示关键字 ASC/DESC 的二进制值。$CONJ 表明连词,表示{and,or}之一。$VAL 表明 WHERE 或 HAVING 条件的值。$SEL 表明 SELECT 语句。

基于槽填充的解码器预测模板中槽位的值,如表 1 所示,根据 SELECT 语句的基本结构,即组成子句的存在和每个子句的条件数。首先组合编码向量 $\boldsymbol{V}^{\mathrm{Q}}$、$\boldsymbol{V}^{\mathrm{D}}$ 和 $\boldsymbol{V}^{\mathrm{P}}$,得到语句编码向量 $\boldsymbol{V}^{\mathrm{S}}$,利用启发式匹配方法拼接函数。通过在 $\boldsymbol{V}^{\mathrm{S}}$ 上应用两个全连接层来分类,得到子句是否存在及每个子句个数的代数表示,必须存在“FROM”和“SELECT”子句才能形成有效的选择语句,表示 INTERSECT、UNION 或 EXCEPT 之一的存在,如果没有这样的子句,则表示 NONE。如果值为 INTERSECT、UNION 或 EXCEPT 之一,则创建相应的 SPC(查询语句位置代码),并递归生成 SELECT 语句结构,生成的 SQL 语句结合 $\boldsymbol{V}^{\mathrm{C}}$、$\boldsymbol{V}^{\mathrm{T}}$ 预测各存在的子任务中填充槽的值,输出生成的 SQL 查询语句。

3.2 逆规范化模式重构

基础模型中出现频率较高的多表连接错误类型如表 2 所示,表中用“pred”表示预测语句,“gold”表示正确语句,示例(a)中多表连接错误体现在表连接顺序颠倒,示例(b)中多表连接错误体现在查询任务可完成的情况下连接多余表,导致不必要的表连接,示例(c)中多表连接错误由多表中含同名列导致表选择错误致使多表连接路径错误,示例(d)中展示了一类特殊类型,虽然 SQL 查询语句有多处错误但查询结果

正确。

表 2　多表连接错误分类与示例

问题类型	错误示例
(a) Join 顺序颠倒	**pred**: SELECT T1. Name, count (*) FROM singer AS T1 JOIN singer_in_concert AS T2 ON T1. Singer_ID = T2. Singer_ID JOIN concert AS T3 ON T2. concert_ID = T3. concert_ID GROUP BY T1. Name
	gold: SELECT T2. name, count (*) FROM singer_in_concert AS T1 JOIN singer AS T2 ON T1. singer_id = T2. singer_id GROUP BY T2. singer_id
(b)多余表连接	**pred**: SELECT T3. Name, T1. Course FROM course AS T1 JOIN course_arrange AS T2 ON T1. Course_ID = T2. Course_ID JOIN teacher AS T3 ON T2. Teacher_ID = T3. Teacher_ID GROUP BY T3. Name ORDER BY T1. Course
	gold: SELECT T3. Name, T2. Course FROM course_arrange AS T1 JOIN course AS T2 ON T1. Course_ID = T2. Course_ID JOIN teacher AS T3 ON T1. Teacher_ID = T3. Teacher_ID
(c) 多表中同含同名列,表选择误差大	**pred**: SELECT T3. Name, count (*) FROM course AS T1 JOIN course_arrange AS T2 ON T1. Course_ID = T2. Course_ID JOIN teacher AS T3 ON T2. Teacher_ID = T3. Teacher_ID GROUP BY T3. Name
	gold: SELECT T2. Name, COUNT(*) FROM course_arrange AS T1 JOIN teacher AS T2 ON T1. Teacher_ID = T2. Teacher_ID GROUP BY T2. Name
(d) 执行结果相同但 sql 语句不同	**pred**: SELECT T2. winner_name FROM players AS T1 JOIN matches AS T2 ON T1. player_id = T2. winner_id WHERE T2. year = 2013 INTERSECT SELECT T2. winner_name FROM players AS T1 JOIN matches AS T2 ON T1. player_id = T2. winner_id WHERE T2. year = 2016
	gold: SELECT winner_name FROM matches WHERE YEAR = 2013 INTERSECT SELECT winner_name FROM matches WHERE YEAR = 2016

基础模型错误案例指出多表连接问题存在巨大改进空间。主外键关系是多表连接问题中不可或缺的部分,在基础模型中,外键关系被用于构建连接路径,逆规范化模式重构组件则对外键信息进行挖掘利用。

数据库模式中对于外键信息用外键名称表示,外键名称由外键标识“FK”与表名组成,如图 3 中 FK 列表所示。由外键关系生成数据库模式集合,如图 3 中 RN 列表所示,对数据库模式集合中的每一组关系做逆规范化处理,得到多组模式,重构以 RN 关系命名,以逆规范化重构模式组加入原数据库模式,投入基础网络。

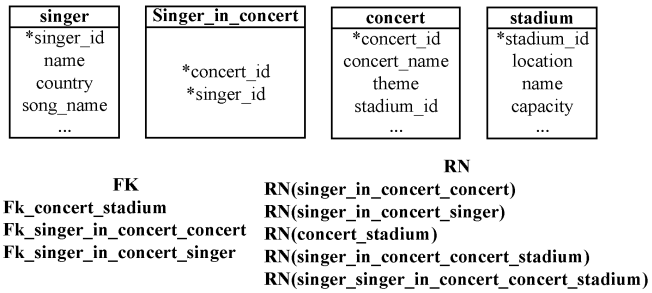

图 3　数据库模式示例

3.3　逆规范化方法

逆规范化技术是一个通过添加冗余数据的数据库优化技术,引用逆规范化技术实施逆规范化模式重构,其目的是通过降低表和表的链接,进而降低外键及索引的数量,引入逆规范化技术可以有效提升检索速率。产生的关系越多,则连接操作就越繁琐,频繁的链接也会影响查找速度。逆规范化技术有如下四种:其一为添加冗余列法,其二为添加派生列法,其三为重新组表法,其四为表分割法。

增加冗余列法即在同一个数据库的多个表中存在相同名称列,这种方法常在查询时避免连接操作。

增加派生列法即将数据库中其他表中的列数据增至当前表中,数据列的选择通常通过计算得到。增加派生法在查询时可避免跨表查询,同时减少多表连接中 JOIN 操作的数目。

重新组表法即将数据库中的两张表重新组成一张单表以减少连接的次数从而提高查询性能。

表分割法即对表做分割,这种方法能够提高查询性能。表分割法通常有两种操作方式:其一为水平分割方式,其操作是将数据库表中一列或多列数据的值进行分割,分割后的数据行分别放入两张独立的表中。其二为垂直分割方式,其应用适合于表中某些列常用而其余列不常用场景下,其操作是将主键及部分列合并到一张表中,将主键及剩余列合并到另一张表中。

数据库逆规范化模式重构将一个数据库中的表按外键关系分成适当个数的区块,对每一个区块中的表群做逆合适的规范化操作,使数据库拥有数张没有外键关系的单表。再将这些单表逐个连接,连成一张单表,完成数据库逆规范化模式重构操作。

数据库模式示例如图 3 示例所示,该示例数据库包含四张表格,分别是 concert 表、stadium 表、singer_in_concert 表、singer 表,数据库模式中存在三组外键,singer 表与 singer_in_concert 表间通过外键 Fk_singer_in_concert_singer 相连,singer_in_concert 表与 concert 表之间通过 Fk_singer_in_concert_concert 相连,concert

表与 stadium 表之间通过外键 Fk_concert_stadium 相连。

外键关系是逆规范化顺序的重要依托指标,按照表连接列表中表名的顺序做逆规范化表连接得到五张单表,如图 4 所示,得到 RN(singer_in_concert_concert)、RN(singer_in_concert_singer)、RN(concert_stadium)、RN(singer_in_concert_concert_stadium)、RN(singer_singer_in_concert_concert_stadium),加入原数据库模式得到重构后的数据库模式 D^C,投入基础模型。

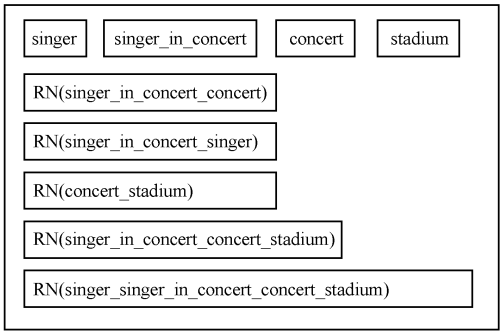

图 4 逆规范化数据库模式图例

4 实验与结果分析

4.1 数据集

实验数据集使用 Spider 数据集,在 Spider 数据集中,训练集、开发集、测试集均不重复,测试集对公众是隐藏的。Spider 数据集于 2018 年被推出,该数据集是在 Text2SQL 数据集中,达到了目前最高难度与复杂度,数据集构成如表 3 所示。

表 3 Spider 数据集构成

类别	#Q	#SQL	#DB
Train	8 695	4 730	140
Dev	1 034	564	20
Test	2 147	—40	

该数据集有以下特点:

1) Spider 数据集涵括丰富的领域来源,其中 SQL 查询语句来自 138 个领域的 200 多个数据库,平均每个数据库包括 5 张数据表,Spider 数据集实现了训练集与测试集的数据库上的区分。

2) SQL 语句更为复杂,Spider 数据集中 SQL 语句的复杂程度呈现在多种维度上,如含有更深的嵌套查询,如含有更多的关键字,如 ORDER BY、UNION、EXCEPT、GROUP BY 等,根据 SQL 查询语句的复杂程度,将 Spider 数据集分为四种难度,从低到高分别为简单(Easy)、中等(Medium)、困难(Hard)以及格外困难(Extra Hard),在如此复杂要求下,为 Text2SQL 任务带来新的挑战。

4.2 评价指标

Spider 数据集提供了两种验证指标—精确匹配(EMA)与执行准确度(EA)。精确匹配指标指的是预测得到的 SQL 查询语句与数据集提供的正确的 SQL 查询语句在各关键字子任务模块上达到字符串完全匹配,而执行准确指标是指,执行预测得到的 SQL 查询语句,使得数据库可以正确执行并且获得一致执行结果。本文的工作以精确匹配率作为 RNSQL 的验证指标更为合理与正确。

4.3 质量评估

与 SQL 查询语句实际生成场景对比,WikiSQL 数据集及 Spider 数据集皆存在大量简化。WikiSQL[17] 是一个大规模的 Text2SQL 语料库,包含来自维基百科的 24 241 个表格上的 80 654 个样本。在 WikiSQL 上,一个问题只与一个表相关,而在 Spider 数据集上,一个问题与多个表相关。

实验在数据集 Spider 上依据子任务划分情况如图 5 所示,列举出五项子任务在单表及多表查询中的训练集样例个数,Spider 数据集在单表与多表、不同任务上的分布不均,本文将验证集划分为单表子集 W1、多表子集 W2、单表多表混合子集 W3。

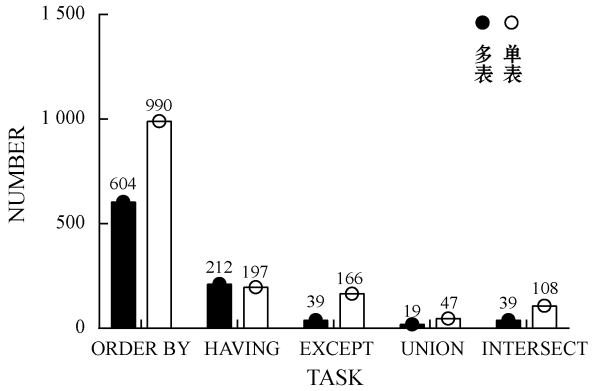

图 5 Spider 训练集表连接个数与对应子任务个数

RNSQL(W3)在 SELECT、SELECT(NO AGG)、WHERE、WHERE(NO OP)、GROUP(NO HAVING)、GROUP、ORDER、AND/OR、IUEN、KEYWORDS 十大任务上的验证结果如图 6 所示,图中 PMA 表示部分匹配精度,PMR 表示部分匹配召回率,PMF 表示部分匹配 F 分数,在 SELECT、AND/OR 及 KEYWORDS 任务上,匹配准确率较高,IUEN 是 INTERSECT、UNION、EXCEPT、NONE 四项子任务的集合,在 IUEN 任务上的准确率较低,其次是 WHERE 任务上的准确率较低。

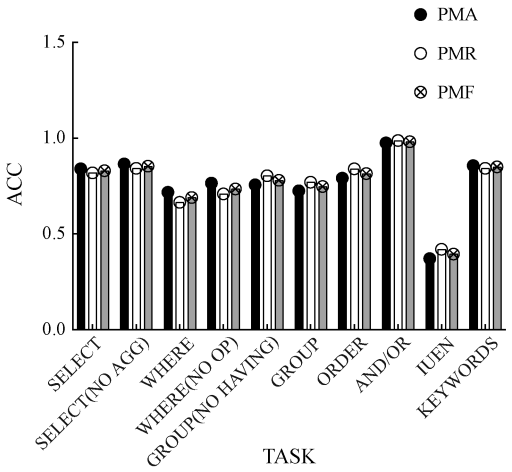

图 6　RNSQL 在子任务上的验证结果

RNSQL(W3)在 Spider 数据集上不同难度子任务上的准确率如表 4 所示,在准确率较高的子任务 SELECT、AND/OR 和 KEYWORDS 上,简单任务的准确率要高于中等难度与困难难度。

表 4　RNSQL(W3)在 Spider 验证集子任务准确率

RNSQL(W3)	EASY	MEDIUM	HARD
SELECT	0.918	0.792	0.917
SELECT(NO AGG)	0.939	0.835	0.917
WHERE	0.844	0.804	0.531
WHERE(NO OP)	0.862	0.826	0.617
GROUP	0.810	0.746	0.739
GROUP	0.810	0.690	0.717
ORDER	0.778	0.720	0.855
AND/OR	1.000	0.982	0.977
IUEN	0.432	0.398	0.373
KEYWORDS	0.917	0.898	0.762

综上所述,RNSQL 模型在验证集上的表现如表 5 所示,相较于 RYANSQL,匹配准确度得到较大提升,在执行准确率上稍逊于 RYANSQL,其根本原因在于数据库模式逆规范化带来的巨大冗余对执行准确造成了巨大干扰,在验证集上实现了 69.65% 的准确率,在精确匹配上同比 RYANSQL 实现约 23% 的提升幅度。

表 5　模型评估

模型	DEV EMA/%
RYANSQL	41.40
RYANSQL(+BERT)	56.50
RNSQL(W1)	70.41
RNSQL(W2)	66.35
RNSQL(W3)	69.65

据式(2)计算得出逆规范化过程综合指标 f 值为 76,所得可证基于逆规范化的 Text2SQL 生成优于基础模型,为多表连接问题带来新的有效的解决方案。

5　结　语

本文针对多表连接问题提出了一种基于逆规范化的 Text2SQL 方法,引入逆规范化技术,重构数据库模式,构建“编码-解码”语义解析器 RNSQL,通过逆规范化将多表连接查询问题简化为单表查询问题,减少表的查找与连接,从而提高查询准确率,模型在 Spider 数据集上进行实验,实验验证 SQL 语句生成质量得到有效提升。引入逆规范化技术为解码过程带来了更复杂的挑战,模式重构带来的部分冗余信息对 SQL 语句执行性能带来隐患,后续将进一步研究逆规范化解码过程的优化以进一步提升模型质量;逆规范化重构模式的选择在本文中是决定模式范围的重点依据,也是冗余信息干扰程度的重要影响因子,故后续将重点研究逆规范化模式重构的模式选择问题。

参　考　文　献

[1] Choi D H, Shin M C, Kim E G, et al. RYANSQL: Recursively applying sketch-based slot fillings for complex text-to-SQL in cross-domain databases[J]. Computational Linguistics,2021,47(2):309 - 332.

[2] Zhen D, Sun S Z, Liu H Z, Lou J G, et al. Data-anonymous encoding for text-to-SQL generation[C]//9th International Joint Conference on Natural Language Processing,2019: 5405 - 5414.

[3] Xu X J, Liu C, Song D, et al. SQLNet: Generating structured queries from natural language without reinforcement learning[EB]. arXiv:1711.04436,2017.

[4] Hwang W, Yim J, Park S, et al. A comprehensive exploration on WikiSQL with table-aware word contextualization [EB]. arXiv:1902.01069,2019.

[5] He P C, Mao Y, Chakrabarti K. X-SQL: Reinforce context into schema representation[EB]. arXiv:1908.08113,2019.

[6] Devlin J, Chang M W, Lee K, et al. Bert: Pre-training of deep bidirectional transformers for language understanding [C]//Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies,2019:4171 - 4186.

[7] Lee D. Clause-wise and recursive decoding for complex and cross-domain text-to-SQL generation[C]//Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing,2019:6047 - 6053.

(下转第 86 页)

for the Brazilian domestic air transportation system[J]. Journal of Air Transport Management,2021,91:101974.

[7] Borsky S, Unterberger C. Bad weather and flight delays: The impact of sudden and slow onset weather events[J]. Economics of Transportation,2019,18:10-26.

[8] 隋东,邢娅萍,涂诗晨. 恶劣天气条件下航路网络修复优化[J]. 航空学报,2021,42(2):323-334.

[9] 顾兆军,安一然,潘杰. 不正常航班旅客流恢复方法研究[J]计算机应用与软件,2016,33(6):79-83.

[10] 况宇琦,赵挺生,蒋灵,等. 塔式起重机事故案例关联规则挖掘与分析[J]. 中国安全科学学报,2021,31(7):137-142.

[11] 李勇男. 基于多层次关联规则挖掘的反恐情报跨层特征关联分析[J]. 情报科学,2021,39(11):127-132.

[12] Wang Z C, Yin J B. Risk assessment of inland waterborne transportation using data mining[J]. Maritime Policy & Management,2020,47(5):633-648.

[13] 吴爱华,陈出新. 分布式数据库中关系数据正负关联规则挖掘[J]. 计算机仿真,2021,38(9):344-347,352.

[14] 索岩,程向羽. 基于贝叶斯后验概率和非合作博弈的推荐算法[J]. 计算机应用与软件,2022,39(3):270-276,284.

[15] 孙智超,王波. 基于数据挖掘技术的建筑能耗分析集成方法[J]. 计算机应用与软件,2017,34(11):103-108,146.

[16] 李航,郭晓梅,胡小兵. 灾害性天气下航空公司天气成本测算模型[J]. 中国安全科学学报,2019,29(6):7-12.

[17] 屈景怡,渠星,杨俊,等. 基于 Skip-LSTM 的机场群延误预测模型[J]. 信号处理,2020,36(4):584-592.

[18] 中国气象服务协会. 短时气象服务降雨量等级: T/CMSA 0013-2019[S]. 北京:中国标准出版社,2019:11.

[19] 中国民用航空局. 航空器机场运行最低标准的制定与实施规定[EB/OL]. [2022-02-27]. https://www.gov.cn/gongbao/content/2002/content_61982.htm.

[20] 黄仪方. 航空气象[M]. 成都:西南交通大学出版社,2011:39-49.

[21] 中国气象局. 降雪的形成条件及等级划分[EB/OL]. [2022-02-27]. http://www.cma.gov.cn/kppd/kppdrt/201311/t20131122_232230.html.

[22] 中国气象局. 风力的等级划分[EB/OL]. [2022-02-27]. http://www.cma.gov.cn/2011xwzx/2011xqxxw/2011xqxyw/201407/t20140718_252716.html.

[23] 中国民用航空局. 一般运行和飞行规则[EB/OL]. [2022-02-27]. http://www.caac.gov.cn/XXGK/XXGK/MHGZ/202202/P020220209518466960506.pdf.

[24] 刘芳兵. 湿滑跑道飞机侧风着陆滑行安全研究[D]. 天津:中国民航大学,2020.

[25] 林甲祥,巫建伟,陈崇成,等. 支持度和置信度自适应的关联规则挖掘[J]. 计算机工程与设计,2018,39(12):3746-3754.

[26] 兰聪花,刘洋,唐占红. AprioriTid 挖掘频繁项集算法的改进[J]. 计算机应用与软件,2010,27(3):234-236.

[27] 崔亮,郭静,吴玲达. 一种基于动态散列和事务压缩的关联规则挖掘算法[J]. 计算机科学,2015,42(9):41-44.

[28] 俞燕燕,李绍滋. 基于散列的关联规则 AprioriTid 改进算法[J]. 计算机工程,2008,34(5):60-62.

[29] 朱文飞,齐建东,洪剑珂. Hadoop 下负载均衡的频繁项集挖掘算法研究[J]. 计算机应用与软件,2016,33(5):35-39.

[30] 白皓. 基于 B/S 结构的搜索管理系统的设计与实现[D]. 北京:北京邮电大学,2020.

(上接第 37 页)

[8] Fu X Y, Shi W J, Yu X D, et al. Design challenges in low-resource cross-lingual entity linking[C]//Conference on Empirical Methods in Natural Language Processing,2020:6418-6432.

[9] Rijhwani S, Xie J T, Neubig G, et al. Zero-shot neural transfer for cross-lingual entity linking[C]//AAAI Conference on Artificial Intelligence,2019,33(1):6924-6931.

[10] Logeswaran L, Chang M W, Lee K, et al. Zero-shot entity linking by reading entity descriptions[EB]. arXiv:1906.07348,2019.

[11] Xu P Y, Hu Q. An end-to-end approach for handling unknown slot values in dialogue state tracking[C]//56th Annual Meeting of the Association for Computational Linguistics,2018:1448-1457.

[12] Nouri E, Hosseini-Asl E. Toward scalable neural dialogue state tracking model[EB]. arXiv:1812.00899,2018.

[13] Herzig J, Berant J. Decoupling structure and lexicon for zero-shot semantic parsing[C]//Conference on Empirical Methods in Natural Language Processing,2018:1619-1629.

[14] Guo J Q, Zhan Z C, Gao Y, et al. Towards complex text-to-SQL in cross-domain database with intermediate representation[C]//57th Annual Meeting of the Association for Computational Linguistics,2020:4524-4535.

[15] Bogin B, Berant J, Gardner M. Representing schema structure with graph neural networks for text-to-SQL parsing[C]//57th Annual Meeting of the Association for Computational Linguistics,2019:4560-4565.

[16] Wang B L, Shin R, Liu X D, et al. RAT-SQL: Relation-aware schema encoding and linking for text-to-SQL parsers[C]//58th Annual Meeting of the Association for Computational Association for Computational Linguistics, Linguistics, 2020:7567-7578.

[17] Zhong V, Xiong C, Socher R. Seq2SQL: Generating Structured Queries from Natural Language using Reinforcement Learning[EB]. arXiv:1709.00103,2017.